# Grooved: Recommendations from the Range Caddy Build

A side-by-side read of Range Caddy and Grooved, written for the Grooved developer. The bias is toward ideas that surfaced *because* Range Caddy was a clean-room rebuild — not feature ports.

Range Caddy paths in this doc are relative to the `slaydenh/range-caddie` repo. Grooved paths are relative to `slaydenh/grooved`.

---

## 1 · Executive summary

- **Grooved's selection pipeline is 8+ ordered sort passes; Range Caddy is one number per drill.** Replacing the layered sorts with a single `score(drill)` is the single largest lever for legibility, testability, and behavioral correctness. See rec #1.
- **`WeaknessProfile.swift` is the right idea, the wrong math.** Net signed integers with session-index linear decay (`1.0 - 0.067 · i`) doesn't carry uncertainty, doesn't reward exploration, and treats a user who plays twice a year the same as one who plays twice a week. Move to a Beta(α, β) posterior per `FocusArea` with calendar-based exponential decay (30-day half-life). See rec #2 and §4.
- **Drop `SessionFeedback.overallRating`.** It is already mid-redesign (`Legacy sessions may have overallRating = 3`, derived from a 60% completion threshold) and adds zero signal on top of per-drill outcomes. See rec #4.
- **Per-drill outcome is binary; that's a free axis you're throwing away.** Replace `Nailed It / Not Yet` with `Struggled / Solid / Dialed` — Range Caddy validated that 3 buttons are tappable one-handed at the range and produces a meaningfully richer Bayesian update than binary. See rec #3.
- **Onboarding is a friction wall before the first real moment of value.** Range Caddy generates a session from zero. Grooved gates a session behind 5 onboarding screens and a profile that the generator only weakly uses. Make all of it optional. See rec #5.
- **The drill swap UI is unused intelligence that invites avoidance.** Swap records persist but never feed back into the generator. The feature also lets users dodge the exact drills the model picked *because* they're hard. See rec #9.
- **You're tracking 6 broad focus areas and ignoring the per-skill detail your library already encodes** (`DrillSubtype`, `supportedMissPatterns`). Both can become Bayesian dimensions cheaply. See §4.

---

## 2 · Feature comparison

| Capability | Range Caddy | Grooved | Observation |
|---|---|---|---|
| Drill library | 40 drills, 15 skill areas, primary/secondary positional weights (`Data/DrillLibrary.swift`) | ~200 drills, 6 `FocusArea` + 11 `DrillSubtype` + 23 `MissPattern` (`MockDrillLibrary.swift`) | Grooved has 5× content depth; RC has finer-grained tagging on a smaller set. |
| Per-drill rating | 3-button: Struggled / Solid / Dialed → 0.10 / 0.55 / 0.92 (`Models/Enums.swift:222`) | 2-button: Nailed It / Not Yet (`StepOutcome`) | Binary outcomes drop the magnitude of struggle — a 3-of-10 drill and a 6-of-10 drill both record `needsWork`. |
| Skill model | Beta(α, β) posterior per skill, time-decayed (`Engine/BayesianModel.swift`) | Net-signed integer-y scores per FocusArea / Subtype / Miss (`Models/WeaknessProfile.swift`) | RC carries uncertainty (stddev); Grooved collapses offsetting evidence to zero. |
| Selection score | One UCB number per skill, weighted into a per-drill score (`Engine/SessionGenerator.swift:87`) | 8+ sequential sort passes inside `generateSession` (`Services/SessionGenerator.swift:59-175`) | Single number is unit-testable; sequential sorts are order-dependent. |
| Setup taps to first drill | 3: time → facility → clubs → Build | 5 onboarding screens, then facility + focus + intent + optional miss | RC: 3 taps from cold install. Grooved: ~10 taps before the first session, ~5 after. |
| Onboarding | None | 5 steps capturing skill / weakness / miss / short-game confidence | Profile is used only softly — skill is a hard filter; weakness/miss give 2× bias that can be dropped. |
| Facility model | 4 mutually-exclusive composite buckets | 3 atomic, multi-select (range + chip + putt) | Grooved's is more correct; the "range + putting, no chip green" case can't be expressed in RC. |
| Club selection | 4 user buckets (putter / wedges / irons / driver) → 8 internal categories | None — drills filtered by FocusArea | RC users can say "30 min, putter + wedges only"; Grooved can't. |
| Time-decay | Exponential, calendar, 30-day half-life on Beta (`BayesianModel.swift:106`) | Linear, session-index: `1 - 0.067 · i` (`WeaknessProfile.swift:38`) | Grooved treats 15 sessions in a week the same as 15 over a year. |
| New-user exploration | UCB stddev term floats unseen skills above 0.5 mean | None — `hasWeaknessSignals` is false; pipeline no-ops for new users | Grooved relies on self-reported weakness for cold start; otherwise blind. |

| Capability | Range Caddy | Grooved | Observation |
|---|---|---|---|
| Drill swap mid-session | Not implemented (dropped in `bd41937`) | Yes — `SwapRecord` persisted "for future generator intelligence" (`Models/GeneratedSession.swift:4`) | Grooved collects swap data but never reads it; the button invites avoidance of hard drills. |
| Pre-round mode | None | `GoalMode.preRoundWarmup` + dedicated slot pipeline (`SessionGenerator.swift:1034`) | Grooved's pre-round path is a clear win RC doesn't match. |
| Brand voice | Minimal — phase labels, three rating words | `KeyThoughtLibrary`, `SessionInsightGenerator`, per-drill `keyThought` / `swingThought` / `whyItHelps` / `hint` | Grooved is a writing product. RC is engineering scaffolding. |
| Persistence | SwiftData (`@Model`) | File-based JSON, `Codable` (`SessionHistoryStore.swift`) | Both work; RC requires iOS 17+. |
| Analytics / monetization | None | PostHog + RevenueCat (free limit 3 sessions) | Grooved correct; RC is pre-revenue. |

---

## 3 · Recommendations, ranked by impact-per-effort

### 1. Replace the 8-layer selection pipeline with a single `score(drill)` function — L

**What.** Today `generateSession` calls, in order: `filteredDrills` → `applyFeedbackWeights` → `applyWeaknessProfile` → `applyShortTermMemory` → `applyWelcomeBackBias` (cond.) → intent prep (`fixAMiss` duplicates, `grooveWhatsWorking` re-sorts, `justFlow` demotes) → `applyOnboardingBias` → interleaving pool prep → `buildSteps` → `Game-Feel Guarantee` → `sortStepsByLocation` → graceful degradation. Each is a `.sorted` or a `matched + matched + pool` duplicate-prepend. The final order depends on `sort` stability and which layer ran last. Collapse to:

```
func score(_ d: Drill, ctx: ScoringContext) -> Double {
    var s = ctx.need(for: d)                  // Bayesian need score
    s += ctx.intentBonus(d)                   // fixAMiss / groove / flow
    s += ctx.onboardingBoost(d)
    s -= ctx.recencyPenalty(d)
    s -= ctx.feltOffPenalty(d)
    return s
}
```

`buildSteps` becomes: rank candidates by `score`, then pick respecting phase + family + duration constraints. The "soft preference" layers (intent, onboarding bias) become weights with named magnitudes instead of duplicate-prepend tricks that depend on downstream dedup.

**Why.** Right now `applyWelcomeBackBias` followed by `justFlow` sort will reorder welcome-back drills again, and the test surface for "welcome-back + justFlow + recent miss" is combinatorial. Single score = unit-testable in isolation.

**Where.** `Services/SessionGenerator.swift` lines 67–175. Touch `buildSteps` line 588 to consume a ranked list instead of a pre-sorted pool.

**Risk.** Behavior changes for every user. Mitigate by porting one layer at a time, locking each behind a coefficient that starts at the current effective weight (e.g. fixAMiss "triple weight" → `+2.0` on `score`), and running `GeneratorFixValidationTests` after each port.

---

### 2. Move `WeaknessProfile` to a Beta(α, β) posterior per FocusArea — M

**What.** Replace `WeaknessProfile.focusAreaScores: [FocusArea: Double]` with `[FocusArea: SkillState]` where `SkillState` mirrors `RangeCaddy/Models/SkillState.swift`:

```
struct SkillState {
    var alpha: Double = 2.0
    var beta:  Double = 2.0
    var lastUpdated: Date = .distantPast

    var mean: Double { alpha / (alpha + beta) }
    var stddev: Double { sqrt(alpha*beta / (pow(alpha+beta, 2) * (alpha+beta+1))) }
}
```

For each completed drill, walk `drill.supportedFocusAreas`, weight by position (1.0, 0.6, 0.4), decay-in-place by calendar half-life, then `alpha += n * r`, `beta += n * (1 - r)` where `r` is the rating's success fraction. Persist alongside `SavedSessionItem` in `sessions.json`, or in a sibling `skill_state.json`.

Replace `weaknessScore(for: drill)` with `need(for: skill) = (1 - mean) + 0.2 * stddev + small_noise`, summed across the drill's primary + secondary focus areas with the same positional weighting.

**Why.** Three failure modes of the current model: (1) new users — `hasWeaknessSignals` is false, the pass no-ops, and the model has nothing to say. The Bayesian version starts at the neutral prior with stddev surfacing untouched skills. (2) Linear session-index decay ignores calendar time — 10 sessions in 14 days and 10 over a year decay identically. (3) Net signed scores collapse offsetting evidence: struggling 5 times and dialing 5 times reads as 0, identical to "no data." `Beta(7, 7)` and `Beta(2, 2)` have the same mean but very different variance — the model should know that.

**Where.** New file `Services/BayesianSkillModel.swift`. Replace `WeaknessProfile.build(from:drillLibrary:)` at `WeaknessProfile.swift:23`. Generator hook is `SessionGenerator.applyWeaknessProfile` at `SessionGenerator.swift:1612` — this becomes a one-line sort by the new `need`.

**Risk.** Hyperparameter retuning needed. Start with RC's values (`baseObservationWeight = 2.0`, `decayHalfLifeDays = 30.0`, `neutralAlpha = neutralBeta = 2.0`) — they were tuned empirically on the same drill semantics. Hold all existing tests; add a "posterior recoverability" test (simulate 5 needsWork ratings on `putting`, assert `need(.putting) > need(.driver)` after).

---

### 3. Promote per-drill outcome to 3 buttons: Struggled / Solid / Dialed — S

**What.** Replace `StepOutcome { completed, needsWork }` with `StepOutcome { struggled, solid, dialed }` and map to success fractions 0.10 / 0.55 / 0.92 exactly as `RangeCaddy/Models/Enums.swift:222`. The dock buttons in `ActiveSessionView.swift:383` become three pills instead of two.

**Why.** Binary outcomes lose magnitude. A drill where the user got 3 of 10 vs 9 of 10 are both "Not Yet" in Grooved today; they tell the posterior radically different stories. From RC commit `eedcbc5`: "a 3-point rating instead of 5 [...] verified end-to-end while implementing the Bayesian skill update." Three is the sweet spot between range-tappable and informative.

**Where.** `Models/SessionStep.swift:3`, dock in `Views/ActiveSessionView.swift:382-443`, recap derive logic in `Models/SessionFeedback.swift:42`. Migrate stored data: existing `.completed` → `.solid`, `.needsWork` → `.struggled`. Add a `StepOutcome.successFraction: Double` property.

**Risk.** A copy / brand decision (the buttons are currently "Nailed It" / "Not Yet" — proud, on-brand). Pair with the copy team. Three labels that hold the same warmth: "Locked it" / "On track" / "Working on it" if "Dialed / Solid / Struggled" reads as too clinical.

---

### 4. Delete `SessionFeedback.overallRating` — S

**What.** Remove the field, the `deriveRating` function, and all weighting logic that branches on it (`SessionGenerator.applyFeedbackWeights` at `SessionGenerator.swift:1571` — drops entirely).

**Why.** Three signs the field is dead code in spirit: 1. The docstring carries legacy compatibility for an already-removed "3 = useful" value (`SessionFeedback.swift:9`). 2. It's *derived* from `drillOutcomes` via a `≥0.6` completion threshold — there's no independent signal. 3. The "skip nil-rated sessions" branch in `applyFeedbackWeights` is dead because every completed session will derive a non-nil rating.

Drill-level outcomes already carry the signal; once #2/#3 above land, this is fully redundant.

**Where.** `Models/SessionFeedback.swift`, `Services/SessionGenerator.swift:1571-1605`, all call sites (mostly tests).

**Risk.** Code churn in tests. Worth it — every removed branch reduces the search space for behavior bugs.

---

### 5. Make onboarding optional — M

**What.** Replace the 5-step `Onboarding*` flow with a single optional "introduce yourself" card on the Home view that surfaces after the user has completed 0 sessions, dismissable. Default `UserProfile` to `name: ""`, `skillLevel: .ninetiesHundred`, `biggestWeakness: nil`. The generator already handles all three being nil (`SessionGenerator.swift:46`: `skillLevel = userProfile?.skillLevel ?? .ninetiesHundred`).

**Why.** RC launches into a real session in 3 taps. Grooved gates that behind ~10 taps of onboarding + form. The profile is also softly used — `biggestWeakness` gives a 2× bias that's silently dropped if it doesn't intersect the per-session focus (`SessionGenerator.swift:1327`). After rec #2 ships the Bayesian model learns the same signal in 3–5 sessions. Brand moments (welcome, name greeting) move into a post-first-session "set yourself up next time" card.

**Where.** `Views/Onboarding*.swift` (gate behind `.optional` navigation, not the initial path). `GroovedApp.swift` initial route becomes Home, not Onboarding.

**Risk.** Loses the brand-onboarding moment (the gold "where do you lose the most shots?" screen is on-voice). Mitigate by keeping the screens, just making them reachable from `SettingsView` and via the opt-in card on Home rather than blocking first use.

---

### 6. Calendar-based exponential decay everywhere — S

**What.** Replace every place Grooved uses session-index decay (`WeaknessProfile.swift:38`) or session-window memory (`SessionGenerator.applyShortTermMemory:1672`) with a calendar half-life.

```
extension SkillState {
    mutating func decayInPlace(now: Date = .now, halfLifeDays: Double = 30) {
        guard lastUpdated != .distantPast else { return }
        let days = now.timeIntervalSince(lastUpdated) / 86_400
        let f = pow(0.5, days / halfLifeDays)
        alpha = neutralAlpha + (alpha - neutralAlpha) * f
        beta  = neutralBeta  + (beta  - neutralBeta)  * f
    }
}
```

**Why.** Practice frequency varies between players by 10× or more. The user notes in `SessionHistoryStore.recentFeedback` say "Memory is measured in sessions, not days — practice frequency varies between players" — that's an argument for *fewer* sessions, not for session count being the right unit. A heavy user's "last 5 sessions" can be a single week; the implicit window should still be ~30 days for both.

**Where.** `Models/WeaknessProfile.swift:38` (decay constant), `Services/SessionHistoryStore.swift:170` (`recentFeedback` window becomes "all feedback in last N days", not "last N sessions").

**Risk.** Existing `28-day welcome-back` detection logic (`SessionGenerator.swift:1637`) overlaps but doesn't collide. Welcome-back can stay as a discrete trigger; calendar decay handles the gradual gap. Keep them both at first; collapse later.

---

### 7. Add a posterior-uncertainty term to skill selection (UCB) — S

**What.** Once #2 ships, the `need` formula already includes `+ 0.2 * stddev`. Make this knob explicit and tunable. The 0.2 coefficient is what stops a Grooved user from never seeing chipping drills if their first session was the range — chipping starts at `stddev = 0.224`, contributing `0.045` to need, enough to lift it above a putting skill the user has rated `mean = 0.85, stddev = 0.05` (need `0.16`).

**Why.** This is the new-user explore-then-exploit guarantee Grooved lacks today. Without it, `WeaknessProfile` is exploit-only — it can only respond to what the user has rated. With it, the model probes untouched skills early and converges to weaknesses as data accumulates.

**Where.** New `BayesianSkillModel.need(for:)` from rec #2.

**Risk.** Coefficient calibration. The current 0.2 was tuned by running rated sessions and checking that rating putting poorly returned putting drills in the next 5 sessions (RC commit `79c5355`). Hold a similar test in Grooved.

---

### 8. Skill-level prior, not skill-level filter — M

**What.** Today `skillLevel` is a hard filter on `drill.supportedSkillLevels` (`SessionGenerator.swift:559`). Use it instead as the *prior* on the Bayesian model. A self-reported `100+` player starts each FocusArea at `Beta(2, 6)` (mean 0.25); a `70s-80s` player starts at `Beta(6, 2)` (mean 0.75). The model still updates from drill outcomes; the prior just shortens cold-start.

**Why.** Hard filtering by skill level throws away ~20–40 % of the library for every user with no compensating signal. A `70s-80s` player who's working on putting basics for a child may want the beginner drill. A `hundredPlus` player who happens to be a former pro at short game shouldn't have advanced chipping drills withheld. Make skill level a soft preference, not a wall.

**Where.** Drop the `drill.supportedSkillLevels.contains(skillLevel)` filter at `SessionGenerator.swift:559`. Replace with an additive score penalty for level mismatch (~−0.1) so it tiebreaks but never excludes. The Bayesian prior change lives in `BayesianSkillModel`.

**Risk.** Some advanced drills genuinely don't make sense for beginners (`MockDrillLibrary` has flop shots, stinger drills). Audit once; keep a `requiredSkillLevel` field for the genuine 10–20 drills where it's true; treat the rest as soft.

---

### 9. Hide the drill swap UI behind a long-press — S

**What.** Make swap a long-press or "..." menu action rather than a visible button on `GeneratedSessionView`. Keep `SwapRecord` persisted (it's data) but stop foregrounding the action.

**Why.** Two problems: (1) swap records are collected but never consumed by the generator — `Models/GeneratedSession.swift:4` says "for future generator intelligence." A feature with no current model effect shouldn't be prominent. (2) Motor learning: the drills the model picks because they're hardest are exactly the ones users swap away. The escape hatch undermines targeted training. Range Caddy dropped its plan-with-swap view in commit `bd41937` for this reason.

**Where.** `Views/GeneratedSessionView.swift`. Replace the inline swap button with a long-press handler on each drill row.

**Risk.** Users complain. Counter: log a `swap_unavailable` analytics event for first 4 weeks before changing; if you see <5 % of users attempting swap on the long-press, you've validated that you can remove it entirely.

---

### 10. Add `successFraction` to `StepOutcome`; remove `recapCategory` fallback — S

**What.** Add to the new 3-state outcome enum:

```
extension StepOutcome {
    var successFraction: Double {
        switch self {
        case .struggled: 0.10
        case .solid:     0.55
        case .dialed:    0.92
        }
    }
}
```

This is the bridge between rec #3 (3-state UI) and rec #2 (Beta update). Today the fallback `Drill.recapCategory` (`Drill.swift:204`) exists to map drills with no FocusArea back to a category, because the recap "What worked" chips need a category to display — but the Bayesian model doesn't; it walks `supportedFocusAreas` directly. Recap chips become a separate UI-only mapping.

**Where.** New extension on `StepOutcome` in `Models/SessionStep.swift`. Keep `recapCategory` for the recap UI; do not let it leak into the model.

**Risk.** Minimal — additive change.

---

### 11. Surface a private skill dashboard, but only after observation count ≥ 3 — M

**What.** Add a `SkillDashboardView` (mirroring RC's deleted one) that shows per-`FocusArea` `mean ± stddev` as horizontal bars. Gate each bar behind `observationCount >= 3` for that FocusArea — if not met, the row shows "Not enough data yet" instead of the neutral 50 % prior bar.

**Why.** RC committed and then removed its skill dashboard (commit `eedcbc5`): "neutral-prior bars at 50% looked like fake data to a new user." The fix isn't to hide the dashboard — it's to hide *rows that don't have data*. This becomes a meaningful "show your work" feature once the Bayesian model is in (rec #2).

**Where.** New view file. Naturally gates behind premium (RevenueCat).

**Risk.** Premature display before #2 ships will mislead users.

---

---

## 4 · Engine deep-dive: the Bayesian skill model

### What it is

For each skill the user might develop, hold a **Beta(α, β) posterior** over a hidden "mastery" parameter θ ∈ [0, 1]. The posterior has two properties that drive everything else:

```
mean  = α / (α + β)              — current estimate of mastery
var   = αβ / ((α+β)² (α+β+1))    — uncertainty
stddev = √var
```

Updates are conjugate. A drill rating contributes a fractional "observation" `r ∈ [0, 1]` with effective sample size `n`:

```
α' = α + n·r
β' = β + n·(1 − r)
```

`r` comes from the rating (RC: `struggled=0.10, solid=0.55, dialed=0.92`). `n` is the drill's importance weight (RC: `baseObservationWeight × positionalWeight × difficultyBoost`), where positional weight is `1.0 / 0.6 / 0.4` for primary / secondary / tertiary skill listed in `drill.supportedFocusAreas`.

Before every update, **decay-in-place** by elapsed calendar time:

```
factor = 0.5 ^ (days_since_update / half_life_days)
α  := neutral_α + (α − neutral_α) · factor
β  := neutral_β + (β − neutral_β) · factor
```

This pulls the posterior gently back toward the neutral prior so the model stays responsive to recent change without erasing history. Half life of 30 days is RC's tuned value.

### Why it's the right tool for this domain

- **Conjugate prior** means O(1) updates; no inference loop, no stored history of every rating. The 6 floats per FocusArea (α, β, observation count, lastUpdated, plus a couple derived) are all you need.
- **Posterior carries uncertainty** — the same number tells you what the model believes *and* how sure it is. This is the explore/exploit budget in a single object.
- **Fractional observations work naturally** with a 3-point rating because Beta is defined on [0, 1] — no awkward bucket-to-pseudocount mapping like "needsWork = +1, completed = −1" (which is what `WeaknessProfile` does today and which collapses informative evidence).
- **Time decay is principled.** Half-life is a single, interpretable hyperparameter that the user could in principle tune ("forget last month" vs "forget the season").

### How to adapt to Grooved's data

Grooved already collects what's needed: `StepOutcome` becomes `r` (after rec #3); positional weight comes from the index in `Drill.supportedFocusAreas`; use `confidenceProfile` as a difficulty proxy until per-drill difficulty exists (`techniqueHeavy = 1.2`, `neutral = 1.0`, `confidenceBuilding = 0.8`); persist `SkillState` per `FocusArea` in a sibling file to `sessions.json`. Don't refactor the six FocusArea taxonomy — see anti-rec §6.1.

**Migration path** — 4 commits, each shippable independently:

1. **C1 — Persist `SkillState` per FocusArea.** Read-only at first; compute from existing `SavedSessionItem` history at app launch. Don't yet hook into the generator.
2. **C2 — Compute `need(for:)` from `SkillState`** and add it to `applyWeaknessProfile` as a tiebreaker behind today's score. Verify in tests that drills targeting weak areas rank ahead.
3. **C3 — Make `need(for:)` the primary score**, drop the old `WeaknessProfile` integer aggregation. Keep the file as a thin wrapper for 1 release for back-compat.
4. **C4 — Decay-in-place on every update**; collapse the welcome-back 28-day detector into the same decay (or keep both — they're not redundant; calendar decay handles gradual cold-start, welcome-back handles a discrete "back from a layoff" feel).

### Extending to subtypes and miss patterns

Grooved's `WeaknessProfile` tracks scores per `DrillSubtype` and per `MissPattern` as well as `FocusArea`. The Bayesian model extends trivially: hold a `SkillState` per `(focusArea, subtype)` or per `MissPattern`. Update each one the drill touches with the same `r`, positional-weighted by the drill's tag.

That said, **don't go to 15 skill dimensions like RC**. RC's `SkillArea` (15 values) was right for a 40-drill library because each drill could be tagged precisely. Grooved's 200-drill library has broader tags; finer skill dimensions would mean re-tagging every drill. Start with 6 `FocusArea` posteriors and revisit only if you see selection failures the model can't explain.

---

## 5 · New ideas — neither app has these

### 5.1 · Bucket-size aware sessions

Range duration maps to a small / medium / large bucket (`PracticeEnums.swift:62` already labels them). Use ball count, not minutes, as the actual session budget for range work. A "small bucket" session has ~50 balls; rep guidance like `"10 balls"` becomes a real budget. Currently both apps suggest a duration and let drills consume it implicitly. Connecting balls-as-currency makes the plan finite in a way users feel.

### 5.2 · Within-block difficulty ramp

Inside a single focus block, sort drills by ascending difficulty. RC has `Drill.difficulty: 1..5` (`Models/Drill.swift:29`); Grooved has `confidenceProfile` as a proxy. The first drill in the block is the easiest version; the last is the hardest. Neither app does this today. It mirrors how a coach actually structures a session: groove, then push.

### 5.3 · Course-context awareness

Grooved already has `userProfile.nextRoundDate` and a round-proximity tier (`SessionGenerator.swift:43`). Push further: let the user paste a course/scorecard URL or pick a course from a list. Generate a session that biases toward shots the course demands (lots of forced carries → driver accuracy; small greens → wedges; bermuda → bump and runs). The data exists in scorecard sources; both apps treat practice as decoupled from the next round.

### 5.4 · Hierarchical skill priors from handicap

A user's self-reported skill level should be a prior on every `SkillState`, not a hard filter. Map handicap to Beta means:

```
hundredPlus       → Beta(2, 6)   mean 0.25
ninetiesHundred   → Beta(3, 4)   mean 0.43
eightiesNineties  → Beta(4, 3)   mean 0.57
seventiesEighties → Beta(6, 2)   mean 0.75
```

The model still updates from outcomes; the prior just gives a head start so the first 5 sessions don't all surface every skill area at equal "need". (See rec #8.)

### 5.5 · Drill graduation

When a drill's pass criteria are met (e.g. Gate Drill 10-in-a-row chained 3 sessions running, indicated by `dialed` ratings), mark it "graduated" and stop scheduling for 30 days. Both apps have implicit versions: RC penalizes recent drills; Grooved hard-excludes from recent sessions. Neither tracks per-drill mastery. Explicit graduation surfaces fresh drills without the user having to dig.

### 5.6 · This-week target

Both apps think in single sessions. Add a weekly intent: "3 sessions this week, total weakness coverage" → distribute focus across sessions. Session 1: putting (your weakest). Session 2: irons. Session 3: pre-round. The Bayesian model already supports this — just let the session planner read "what have I covered this week" and rotate.

### 5.7 · Confidence-targeted finisher

Both apps end with a finisher / cool-down. Make the finisher dynamic: pick a confidence-rebuilder if the session has been mostly `struggled` ratings; pick a pressure closer if the session has been mostly `dialed`. Both apps already have the metadata (`confidenceProfile`, `sessionRole: .finisher`); neither uses in-session signal to choose.

### 5.8 · Apple Watch + speech rating

"Hey Siri, that drill was solid." Both apps require pulling the phone out between drills. The Watch + speech rating combination removes the last bit of friction at the range. RC's roadmap mentions it; Grooved should too. Higher-effort but high-impact for the bag-of-balls workflow.

### 5.9 · Consolidation bump

Motor learning research shows skill gains consolidate overnight. Apply a small positive nudge to the posterior 24–48 h after a session: `α += 0.2`, `β -= 0.1`. The user logs in tomorrow and the model has "learned" a tiny bit on its own, reflecting the user's brain doing the same. Tunable; minor; differentiates from every other practice app on the market.

---

## 6 · Anti-recommendations

### 6.1 · Don't copy Range Caddy's 15 skill dimensions

RC tracks `puttingShort`, `puttingMid`, `puttingLag`, `puttingBreak`, `chippingGreenside`, `chippingPitch`, `wedgePartial`, `ironShort`, `ironMid`, `ironLong`, `woodsHybrid`, `driverAccuracy`, `driverDistance`, `mentalPressure`, `transferRandom`. This was tractable for a 40-drill library tagged from scratch. Grooved has ~200 drills tagged across 6 `FocusArea` × 11 `DrillSubtype` × 23 `MissPattern`. Going to 15 fine-grained skills means re-tagging every drill. The Bayesian model works fine on 6 dimensions; the variance term provides exploration even at coarse granularity.

### 6.2 · Don't drop onboarding entirely

RC has zero onboarding because it has no brand. Grooved's onboarding includes the gold "where do you lose the most shots?" moment — that *is* the brand. Make onboarding optional (rec #5), not absent. Lose the welcome-screen → continue → continue → continue chain; keep the single most memorable screen.

### 6.3 · Don't drop drill-time tracking

RC doesn't track `actualSecondsSpent`. Grooved does (`SessionStep.swift:18`). That data feeds future "you take longer on chipping than you think" insights and helps tune drill duration defaults. Keep it.

### 6.4 · Don't drop the brand voice in drill content

RC's drill `instructions` are workmanlike: `"Hit 10 putts from 6 ft."` Grooved's are voiced: `"Most driver misses start offline before curve even matters."` plus `keyThought`, `swingThought`, `whyItHelps`, `hint`. That voice is why a user installs Grooved instead of building a session in Notes. Don't trade brand for engine simplicity — they're not in tension.

### 6.5 · Don't drop multi-facility selection

RC simplified to four mutually-exclusive facility *buckets* (Full / Range / Short / Putt). Grooved's atomic multi-select (range + chipping + putting, any subset) is more correct — the common "range + putting, no chipping green at this place" configuration can't be expressed in RC. Keep your facility model.

### 6.6 · Don't move to SwiftData

RC uses SwiftData (`@Model`). Grooved uses file-based JSON. SwiftData locks you to iOS 17+; the schema migration story is rough; the `@MainActor` requirements ripple. Grooved's JSON-on-disk approach is boring and bulletproof. Stay there.

### 6.7 · Don't drop the rich per-drill content schema

Grooved's `Drill` has `keyThought`, `swingThought`, `whyItHelps`, `hint`. RC has none of these. The cue card in `ActiveSessionView.cueAnchor` is half the on-range value of the app. Keep all four fields.

---

*End.*